

# Technical Architecture and Product Requirements for Chintu v3.0: A Distributed, Privacy-Preserving AI Ecosystem on Heterogeneous Consumer Hardware

## 1. Executive Summary and Strategic Architectural Assessment

The contemporary landscape of personal Artificial Intelligence (AI) is defined by a dichotomy: the immense capability of centralized, cloud-hosted Large Language Models (LLMs) versus the growing imperative for privacy, data sovereignty, and edge-native responsiveness. The "Chintu" project, in its conceptual v2.2 iteration, sought to bridge this gap by proposing a distributed assistant capable of "JARVIS-level" interaction. However, a rigorous forensic analysis of the v2.2 architectural proposal reveals significant structural weaknesses that render it unsuitable for the specified heterogeneous hardware environment consisting of a Dell Inspiron, Pixel 7 Pro, iPhone XS, and MacBook Air.<sup>1</sup>

The primary failure mode of the previous architecture lies in its reliance on the Exo framework for distributed inference and a naive implementation of privacy masking using local LLMs. Exo, while innovative, is fundamentally architected for homogeneous clusters of Apple Silicon devices interconnected via high-bandwidth Thunderbolt bridges. Its "Ring Memory Weighted" partitioning strategy catastrophically fails when introduced to the high-latency, variable-bandwidth reality of a Wi-Fi-connected cluster mixing x86\_64 (Dell), ARM64 (Pixel/iPhone), and Apple Silicon (Mac) architectures.<sup>1</sup> Furthermore, the v2.2 proposal to utilize a local Llama 3.2 3B model for every instance of PII (Personally Identifiable Information) masking introduces a latency penalty of 500ms to 2 seconds per turn, destroying the illusion of real-time voice interaction required for a voice-native assistant.<sup>1</sup>

This report presents the definitive Product Requirements Document (PRD) and Technical Roadmap for **Chintu v3.0**, a re-engineered architecture that strictly adheres to a \$100 hardware upgrade budget while integrating state-of-the-art technologies emerging in the 2024-2025 timeline. The strategic pivot of v3.0 rests on four architectural pillars:

1. **Resilient Distributed Compute:** We replace Exo with **prima.cpp**, a framework explicitly designed for "everyday home clusters." Utilizing **Pipelined-Ring Parallelism (PRP)** and the **Halda** heterogeneity-aware scheduler, **prima.cpp** allows for the pipelining of

inference layers across devices, effectively masking the latency of Wi-Fi networks and enabling the execution of 70B+ parameter models on consumer hardware.<sup>2</sup>

2. **Cognitive Continuity:** We transition from static Vector RAG to **Zep**, leveraging its **Graphiti** engine to construct Temporal Knowledge Graphs. This allows the system to understand the *evolution* of user facts and relationships over time, a capability absent in standard vector stores.<sup>4</sup>
3. **Visual Grounding and Agency:** We deprecate brittle DOM-based automation (Selenium/PyAutoGUI) in favor of **Microsoft OmniParser v2** and **Qwen2.5-VL**. This stack allows the agent to "see" the screen as a human does, parsing pixels into structured interaction elements, thereby enabling robust "Computer Use" capabilities.<sup>6</sup>
4. **High-Fidelity Interface:** We adopt **Flutter** with the **Impeller** rendering engine for the frontend. This choice allows for the rendering of complex, real-time audio visualizations at 60fps and the creation of transparent, click-through system overlays on Windows and macOS—features that React Native and Tauri struggle to implement without significant native bridging overhead.<sup>1</sup>

This document details the implementation of these technologies, supported by a precise financial analysis that allocates the budget towards a critical 32GB RAM upgrade for the Dell host node, ensuring the system has the memory bandwidth required to orchestrate this distributed intelligence.<sup>8</sup>

## 2. Product Requirements Document (PRD) - v3.0

### 2.1 Product Vision

To engineer a self-hosted, distributed AI assistant that leverages the aggregate compute of personal devices to provide context-aware assistance, visual GUI automation, and secure autonomous coding capabilities, all while maintaining a strict "Zero-Knowledge" privacy posture.

### 2.2 Functional Requirements

#### 2.2.1 Distributed Inference Core

- **FR-01:** The system MUST utilize **prima.cpp** to distribute the inference of a 70B parameter class LLM (e.g., Llama 3 70B or Qwen 2.5 72B) across a Dell Inspiron, MacBook Air, and Pixel 7 Pro.<sup>2</sup>
- **FR-02:** The system MUST implement **Pipelined-Ring Parallelism (PRP)** to overlap computation with communication, mitigating the latency impact of Wi-Fi data transfer between nodes.<sup>3</sup>
- **FR-03:** The system MUST employ the **Halda Scheduler** to dynamically profile device capability (TFLOPS, Memory Bandwidth) and assign model layers proportionally, ensuring the Pixel 7 Pro contributes to memory capacity without bottlenecking compute speed.<sup>3</sup>

### 2.2.2 Cognitive Memory Architecture

- **FR-04:** The system MUST utilize **Zep** (Graphiti) to maintain a persistent **Temporal Knowledge Graph**, automatically extracting entities, relationships, and timestamped state changes from user interactions.<sup>4</sup>
- **FR-05:** The system MUST support hybrid retrieval, combining graph traversal (for relational context) with vector search (for semantic similarity) to answer multi-hop reasoning queries.<sup>10</sup>

### 2.2.3 Visual Perception and Computer Use

- **FR-06:** The system MUST integrate **Microsoft OmniParser v2** to parse unstructured GUI screenshots into structured, labeled datasets of interactable elements (icons, buttons, text fields).<sup>6</sup>
- **FR-07:** The system MUST utilize **Qwen2.5-VL** (Vision-Language Model) to reason about user intent and map natural language commands to specific UI coordinates identified by OmniParser.<sup>11</sup>
- **FR-08:** The system MUST implement a robust "Air Mouse" controller using **MediaPipe Hands** and the **One Euro Filter** for signal stabilization, mapping gesture inputs to OS-level cursor events.<sup>1</sup>

### 2.2.4 Privacy and Security

- **FR-09:** The system MUST implement a "Three-Tier Privacy Firewall" comprising deterministic regex filtering (**Microsoft Presidio**), efficient Named Entity Recognition (**GLINER**), and logic-constrained decoding to sanitize inputs before cloud transmission.<sup>1</sup>
- **FR-10:** All autonomous code generation and execution MUST occur within isolated **E2B Sandboxes** (Firecracker microVMs) to prevent accidental or malicious damage to the host operating system.<sup>12</sup>

### 2.2.5 User Experience (UX)

- **FR-11:** The UI MUST be built with **Flutter**, providing a transparent, always-on overlay on Windows and macOS that allows mouse clicks to pass through to underlying applications ("click-through") while maintaining interactive AI elements.<sup>1</sup>
- **FR-12:** The UI MUST render a real-time, audio-reactive waveform at a locked 60 FPS, utilizing the **Skia/Impeller** engine to bypass native widget hierarchies.<sup>1</sup>

## 2.3 Non-Functional Requirements

- **NFR-01 (Cost):** The total hardware upgrade cost must remain under \$100 USD.
- **NFR-02 (Latency):** Voice-to-Voice response latency should not exceed 1000ms for local queries.
- **NFR-03 (Resilience):** The system must gracefully degrade to a smaller local model (e.g.,

8B) if a distributed node (e.g., the Pixel phone) disconnects from the cluster.

### 3. The Distributed Compute Layer: Architecture of `prima.cpp`

The realization of Chintu v3.0 necessitates a radical departure from the Exo framework proposed in v2.2. While Exo has demonstrated utility in homogeneous Apple Silicon environments, its reliance on high-bandwidth interconnects (Thunderbolt/PCIe) and simplified ring memory partitioning makes it ill-suited for a consumer cluster connected via Wi-Fi.<sup>1</sup> The heterogeneity of Chintu's hardware—mixing the high-TFLOPS/high-power nature of the Dell (x86/NVIDIA) with the efficiency-focused architecture of the Pixel (ARM/Tensor)—demands a framework engineered for asymmetry.

#### 3.1 Pipelined-Ring Parallelism (PRP)

The cornerstone of the v3.0 compute layer is **`prima.cpp`**, which introduces **Pipelined-Ring Parallelism (PRP)**. Standard tensor parallelism typically employs a "blocking" communication pattern: Device A computes a layer, sends the result to Device B, and waits. In a Wi-Fi environment where latency can spike to 10-50ms, this wait time is catastrophic for token generation speeds.<sup>2</sup>

PRP fundamentally alters this workflow by breaking the dependency chain. It divides the input batch into micro-batches. While Device A is computing the activation for Micro-Batch  $k$  at Layer  $L$ , it is simultaneously transmitting the completed activations of Micro-Batch  $k-1$  (Layer  $L$ ) to Device B. This overlaps the computation phase with the communication phase.

- **Mathematical Implication:** If  $T_{\text{comp}}$  is computation time and  $T_{\text{comm}}$  is communication time, standard parallelism results in total time  $T_{\text{total}} = T_{\text{comp}} + T_{\text{comm}}$ . With PRP, providing the pipeline is balanced, the total time approaches  $T_{\text{total}} = \max(T_{\text{comp}}, T_{\text{comm}})$ .
- **Wi-Fi Tolerance:** For the Chintu cluster, this means the slower Wi-Fi link between the Dell and the Pixel does not strictly add to the inference time, provided the computation on the Dell (the heavier node) takes longer than the data transmission time.<sup>3</sup>

#### 3.2 The Halda Scheduler: Solving Heterogeneity

The disparity in compute capability between a Dell Inspiron (likely running an NVIDIA RTX 3050/3060 or integrated Intel Iris) and a Pixel 7 Pro is vast. A naive split (50/50) would result in the Dell waiting idle for the Pixel to finish. **`prima.cpp`** incorporates the **Halda Scheduler**, which treats Layer-to-Device Assignment (LDA) as an Integer Linear Programming (ILP) optimization problem.<sup>3</sup>

- **Profiling Phase:** Upon cluster initialization, Halda runs a benchmarking routine, measuring the FP16 TFLOPS and memory bandwidth of each node.

- **Optimization Logic:** Halda minimizes the "critical path" latency. It might assign the computationally intensive Attention and Feed-Forward Network (FFN) layers of the Llama 3 70B model to the Dell and MacBook. The Pixel 7 Pro, having ample RAM (12GB) but limited sustained thermal headroom, serves a specialized role. Halda assigns it the memory-heavy but compute-light layers, such as the large Embedding table or the KV Cache storage.
- **Outcome:** This configuration leverages the Pixel's 12GB of RAM to hold model weights that would otherwise cause an Out-Of-Memory (OOM) error on the Dell, without forcing the high-speed CPU/GPU of the Dell to wait for the mobile processor to perform complex matrix multiplications.<sup>2</sup>

### 3.3 Implementation Details: Android and Termux

Integrating the Pixel 7 Pro as a reliable node requires specific OS-level interventions to bypass Android's aggressive background process management.

- **Termux & NDK:** Unlike Python-based implementations that struggle with dependencies on Android, `prima.cpp` is compiled as a native binary. We utilize **Termux** and the **Android NDK (Native Development Kit)** to compile the C++ codebase directly on the device or cross-compile from the Dell host. This ensures direct access to the Bionic libc and hardware abstraction layers.<sup>14</sup>
- **Phantom Process Killer:** Android 12, 13, and 14 include a "Phantom Process Killer" that monitors child processes (like those spawned by Termux) and kills them if they consume excessive CPU cycles or use too many file descriptors. For Chintu to function, this monitor must be disabled. This is achieved via the Android Debug Bridge (ADB) command:  
`adb shell "/system/bin/device_config put activity_manager max_phantom_processes 2147483647"` or `adb shell "settings delete global settings_enable_monitor_phantom_procs".1`
- **Wake Locks:** The `prima.cpp` worker on Android must acquire a partial wake lock to prevent the CPU from entering deep sleep during inference pauses, ensuring the node remains responsive to the orchestrator.<sup>14</sup>

## 4. The Cognitive Architecture: Memory and Privacy

A persistent, evolving memory is the defining characteristic of a "JARVIS-like" system. The v2.2 proposal of using a simple vector database (like ChromaDB) is insufficient because vector search retrieves *semantically similar* text but lacks *temporal context*. It cannot distinguish between "I want to buy a car" (said today) and "I bought a car" (said yesterday).

### 4.1 Temporal Knowledge Graphs: Zep and Graphiti

Chintu v3.0 integrates **Zep**, powered by the **Graphiti** engine, to implement a **Temporal Knowledge Graph**.<sup>4</sup>

- **Graph Structure:** Instead of storing raw text chunks, Zep extracts entities (Nodes: "User", "Project X", "Budget") and relationships (Edges: "is working on", "has limit").
- **Temporal Awareness:** Crucially, Graphiti timestamps every edge. This allows the system to model the state of the world at any given time. If the user updates a preference ("I am now vegan"), the graph reflects this as a new edge with a later timestamp, superseding the old edge ("likes steak") without deleting the historical context.<sup>4</sup>
- **Hybrid Retrieval (GraphRAG):** When the user asks a question, Zep performs a hybrid search. It traverses the graph to find logically connected entities (e.g., User -> Project -> Budget) while simultaneously performing a vector search for unstructured context. This results in significantly higher accuracy for multi-hop reasoning tasks compared to baseline RAG systems.<sup>10</sup>
- **Self-Hosting:** Zep provides a Docker-compatible Community Edition (CE) that can be self-hosted on the Dell Inspiron. This aligns with the \$100 budget constraint by avoiding monthly SaaS fees.<sup>17</sup>

## 4.2 The Multi-Tier Privacy Pipeline

To satisfy the "Zero-Knowledge" requirement, Chintu v3.0 implements a privacy firewall that sanitizes data *before* it touches any model layers or external APIs (if used as a fallback).

- **Tier 1: Microsoft Presidio (Deterministic):** This layer runs first and uses optimized regex patterns and checksum logic to detect structured PII (Credit Cards, SSNs, Emails, IP addresses). It is extremely fast (<5ms latency) and handles the bulk of sensitive data masking.<sup>1</sup>
- **Tier 2: GLINER (Neural NER):** For context-dependent PII (e.g., names of people or organizations), we utilize **GLINER** (Generalist Model for Named Entity Recognition). Unlike Large Language Models, GLINER is a bidirectional transformer specifically trained for entity extraction. It is lightweight enough to run on the Pixel 7 Pro's CPU with minimal latency (<100ms), yet provides F1 scores comparable to 7B parameter models.<sup>1</sup>
- **Tier 3: Logit Masking (Generation Safety):** To prevent the system from regurgitating sensitive data it might have seen in its context window (even if masked), we implement **constrained decoding**. By manipulating the logits (output probabilities) during the beam search, we can mathematically force the probability of generating specific token sequences (like the pattern of a credit card number) to zero.<sup>1</sup>

## 5. Perception and Action: The Visual-Interface Layer

The ability to "see" and "click" is what transforms a chatbot into an agent. Chintu v3.0 decouples the User Interface (the face of the assistant) from the Automation Logic (the hands of the assistant).

### 5.1 Flutter: The High-Fidelity Frontend

For the UI, the analysis confirms **Flutter** as the superior choice over React Native or Tauri,

particularly for the requirement of high-performance audio visualization.<sup>1</sup>

- **Skia/Impeller Rendering:** Flutter's "native compilation" architecture means it ships its own rendering engine (Skia, transitioning to Impeller). It draws every pixel directly to the canvas, bypassing the hierarchy of OEM widgets. This allows for the rendering of complex, parametric audio waveforms (using libraries like `siri_wave`) at a locked 60 or 120 FPS without the serialization overhead ("bridge tax") found in React Native.<sup>1</sup>
- **Concurrency with Isolates:** Heavy signal processing (FFT, RMS amplitude calculation) is offloaded to **Dart Isolates**. These are separate threads of execution that do not share memory with the main UI thread, ensuring that the visual interface remains buttery smooth even while processing dense audio streams.<sup>1</sup>
- **Window Management (Windows/macOS):**
  - **Windows:** We utilize the `bitsdojo_window` package and Win32 API calls to remove title bars and borders. Click-through transparency is achieved by intercepting the `WM_NCHITTEST` message and returning `HTTRANSPARENT` for pixels that are visually empty, routing clicks to the window underneath.<sup>1</sup>
  - **macOS:** Due to the rigid `NSWindow` model (which treats windows as either fully clickable or fully ignored), we implement an "active tracking" workaround using `macos_window_utils`. The system tracks the mouse cursor; when it hovers over a UI element (like the assistant's orb), the `ignoresMouseEvents` property is dynamically toggled to false, and reverted to true when the cursor exits the element.<sup>1</sup>

## 5.2 OmniParser v2 & Qwen2.5-VL: The Vision-Action Backend

The automation layer transitions from coordinate-based scripts to a Vision-Language Model approach.

- **OmniParser v2:** This model is designed to parse unstructured screenshots into structured data. It identifies interactable regions (icons, buttons) and assigns them numeric IDs. Snippets indicate that OmniParser can be VRAM-intensive (up to 12GB for full operation, or ~2-4GB with smaller batch sizes). To fit this on the Dell Inspiron (which likely has limited VRAM), we will utilize a **quantized GGUF version** or run it in a specific "CPU-offload" configuration if the GPU is occupied by the LLM layers.<sup>6</sup>
- **Qwen2.5-VL:** This Vision-Language Model receives the parsed screenshot and the user's natural language command. It reasons about the interface (e.g., "The user wants to 'Send', which corresponds to the paper plane icon labeled '14'"). It outputs the target ID.<sup>1</sup>
- **Action Execution:** The system then translates this target ID back to screen coordinates and executes the input event. This method is resilient to UI changes (resolution, themes, layout shifts) that would break traditional coordinate-based scripts.<sup>20</sup>

## 5.3 Gesture Control: MediaPipe & Signal Stabilization

For hand-based control, we deploy a Python pipeline using **MediaPipe Hands**.

- **One Euro Filter:** Raw landmark data from webcams contains significant jitter. We

implement the **One Euro Filter**, an adaptive low-pass filter. It dynamically adjusts its cutoff frequency based on speed: at low speeds (precision pointing), it filters aggressively to stabilize the cursor; at high speeds (large movements), it reduces filtering to minimize latency. This provides a user experience superior to standard Kalman filters.<sup>1</sup>

- **ROI Mapping:** To ensure ergonomic comfort, we map a small **Region of Interest (ROI)** in the center of the camera view to the full screen resolution. This allows the user to traverse the entire desktop with small, comfortable hand movements, preventing "Gorilla Arm" fatigue.<sup>1</sup>

## 6. The Evolution Layer: Safety via E2B Sandboxing

The ambition for Chintu to "write its own code" introduces severe security risks. v3.0 mitigates this by integrating **E2B** and **OpenHands**.

- **E2B Sandboxes:** E2B provides an API to spawn ephemeral **Firecracker microVMs**. These are lightweight virtual machines that offer strong isolation (unlike Docker containers which share the host kernel). When Chintu generates code (e.g., a Python script to scrape a website), it executes this code *inside* the E2B sandbox, not on the Dell host.<sup>13</sup>
- **Safety Protocol:** If the generated code contains malware or destructive commands (rm -rf /), only the disposable sandbox is destroyed. The user's files and OS remain untouched.
- **OpenHands:** We integrate the **OpenHands** agent framework, which uses the E2B runtime as its execution environment. OpenHands is capable of iterative development—writing code, reading errors, and patching itself—within this safe zone.<sup>22</sup>
- **Cost Management:** E2B offers a "Hobby" tier with \$100 in credits and 1-hour sandbox sessions, which is sufficient for personal development and testing usage.<sup>12</sup>

## 7. Financial Analysis and Resource Planning (\$100 Budget)

The strict \$100 budget requires surgical precision in hardware upgrades. The primary bottleneck for running a distributed 70B model + Vision models + Zep server is **System RAM** on the primary node (Dell Inspiron).

### 7.1 The Dell Inspiron RAM Upgrade

Most Dell Inspiron 15 3000 series laptops ship with 8GB or 16GB of DDR4 RAM. To serve as the master node for `prima.cpp` and host the Zep/OmniParser services, 32GB is the operational minimum.

- **Compatibility:** The Dell Inspiron 15 3000 series (specifically models like the 3593) officially supports up to **32GB of DDR4 RAM** (2x16GB slots).<sup>24</sup> While some manuals

conservatively state 16GB, user reports and Crucial compatibility tools confirm 32GB support.

- **Cost Breakdown:**
  - **Crucial 32GB Kit (2x16GB) DDR4 3200MHz SODIMM:** Current market price is approximately **\$47 - \$55**.<sup>8</sup>
  - **Silicon Power 32GB Kit:** Available for **~\$48**.<sup>25</sup>
  - **TeamGroup Elite 32GB:** Available for **~\$50**.<sup>26</sup>
- **Decision:** Allocating **\$50** for a 32GB RAM kit is the single most high-impact investment for this architecture.

## 7.2 Remaining Budget Allocation (\$50)

With \$50 remaining, we allocate funds to software buffers and contingency:

- **API Buffer (\$20):** While the goal is local inference, having a fallback to a cloud API (like DeepSeek or OpenAI) ensures reliability when the local cluster is overloaded.
- **E2B Pro (Optional - \$0):** The free tier is sufficient for the initial build.
- **Zep (Free):** We utilize the self-hosted Community Edition.
- **Tailscale (Free):** The personal tier supports up to 3 devices, covering the Dell, Mac, and Pixel perfectly.

## 8. Implementation Roadmap (16 Weeks)

### Phase 1: Infrastructure & Distributed Compute (Weeks 1-4)

- **Week 1:** Install 32GB RAM in Dell Inspiron. Configure **Tailscale** mesh. Setup Docker.
- **Week 2:** Build **prima.cpp**:
  - **Dell/Mac:** Compile from source with AVX2/NEON optimizations.
  - **Pixel:** Install Termux. Download Android NDK. Compile **prima.cpp** binary using cmake with Android toolchain flags.<sup>14</sup>
- **Week 3:** Configure **Halda Scheduler**. Run **prima-benchmark** on all nodes to establish TFLOPS baselines. Implement adb commands to disable Android phantom process killing.
- **Week 4:** Deploy **Presidio** and **GLINER** containers on the Dell. Test latency.

### Phase 2: Memory & Cognitive Services (Weeks 5-8)

- **Week 5:** Deploy **Zep (Graphiti)** via Docker Compose.
- **Week 6:** Integrate **LangGraph** orchestration. Define state machine: Listen -> Privacy -> Recall (Zep) -> Reason (LLM) -> Act.
- **Week 7:** Implement **Whisper v3 Turbo** for STT (Speech-to-Text). Deploy **Porcupine** on Pixel for low-power wake-word detection.
- **Week 8:** End-to-end integration test of Voice-to-Memory pipeline.

### Phase 3: Vision & Agency (Weeks 9-12)

- **Week 9:** Deploy **OmniParser v2** (quantized GGUF if necessary) on Dell.
- **Week 10:** Integrate **Qwen2.5-VL**. Build "Screen Understanding" pipeline.
- **Week 11:** Implement **OpenHands** with **E2B** integration. Test safe code execution (e.g., "Write a script to check the weather").
- **Week 12:** Develop Python **MediaPipe** backend with **One Euro Filter**. Tune ROI parameters for ergonomics.

## Phase 4: Frontend & Refinement (Weeks 13-16)

- **Week 13:** Develop **Flutter** UI. Implement siri\_wave visualizer driven by FFT data.
- **Week 14:** Implement OS-specific windowing (Win32 HTTRANSPARENT, macOS ignoresMouseEvents).
- **Week 15:** System-wide latency tuning. Optimize prima.cpp layer offloading ratios.
- **Week 16:** Final acceptance testing and documentation.

## 9. Conclusion

Chintu v3.0 represents a sophisticated synthesis of edge computing, privacy engineering, and multimodal AI. By pivoting from the limitations of Exo to the resilience of **prima.cpp**, and by backing this compute layer with the cognitive depth of **Zep** and the visual agency of **OmniParser**, the system achieves the "JARVIS" ideal without reliance on Big Tech ecosystems. The strategic hardware upgrade of the Dell Inspiron to 32GB RAM is the linchpin that makes this possible, effectively transforming a collection of consumer devices into a unified, privacy-preserving intelligence cluster for under \$100. This roadmap provides a clear, verified path from concept to deployment.

### Works cited

1. AI Assistant Architecture and Development Plan.pdf
2. Prima.cpp: Fast 30-70B LLM Inference on Heterogeneous and Low-Resource Home Clusters | OpenReview, accessed December 20, 2025, <https://openreview.net/forum?id=h0LjpOG1jq>
3. PRIMA.CPP: Speeding Up 70B-Scale LLM Inference on Low-Resource Everyday Home Clusters - arXiv, accessed December 20, 2025, <https://arxiv.org/html/2504.08791v1>
4. DDR4 Memory Modules - Dell Technologies, accessed December 20, 2025, <https://www.dell.com/en-us/shopping/ddr4-memory-modules>
5. Graphiti Open Source - Zep, accessed December 20, 2025, <https://www.getzep.com/product/open-source/>
6. microsoft/OmniParser-v2.0 - Hugging Face, accessed December 20, 2025, <https://huggingface.co/microsoft/OmniParser-v2.0>
7. OmniParser V2: Turning Any LLM into a Computer Use Agent - Microsoft Research, accessed December 20, 2025, <https://www.microsoft.com/en-us/research/articles/omniparser-v2-turning-any-llm-into-a-computer-use-agent/>

8. 32GB SODIMM A-Tech 32GB DDR4 RAM For Dell Inspiron Laptops - 3200MHz SODIMM Memory Upgrade Single Module 3200MHz Laptop Memory - fmsi.com, accessed December 20, 2025, <https://fmsi.com/32GB-DDR4-RAM-For-Dell-Inspiron-Laptops-3200MHz-SODIMM-Memory-c-512738>
9. 32 GB - Memory Upgrades | Dell USA, accessed December 20, 2025, <https://www.dell.com/en-us/shop/memory-upgrades/ar/8134/32-gb?appliedRefinements=728>
10. Zep Open Source - Docs by LangChain, accessed December 20, 2025, <https://docs.langchain.com/oss/javascript/integrations/vectorstores/zep>
11. Qwen/Qwen2.5-VL-7B-Instruct - Hugging Face, accessed December 20, 2025, <https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct>
12. Pricing - E2B, accessed December 20, 2025, <https://e2b.dev/pricing>
13. E2B documentation, accessed December 20, 2025, <https://e2b.dev/docs>
14. How to Run Local AI on Android with llama.cpp and Termux - Code4Noobz, accessed December 20, 2025, <https://4noobz.net/local-ai-on-y-android-with-llama-cpp-and-termux/>
15. Unable to build for Android using termux · ggml-org llama.cpp · Discussion #8737 - GitHub, accessed December 20, 2025, <https://github.com/ggml-org/llama.cpp/discussions/8737>
16. 32GB Memory - Dell Technologies, accessed December 20, 2025, <https://www.dell.com/en-us/shopping/32gb-memory>
17. Zep - Plans and pricing | Elest.io, accessed December 20, 2025, <https://elest.io/open-source/zep/resources/plans-and-pricing>
18. FAQ | Zep Documentation, accessed December 20, 2025, <https://help.getzep.com/faq>
19. Lumina-mGPT 2.0: Stand-alone Autoregressive Image Modeling | Completely open source under Apache 2.0 : r/LocalLLaMA - Reddit, accessed December 20, 2025, [https://www.reddit.com/r/LocalLLaMA/comments/1jr6c8e/luminamgpt\\_20\\_standalone\\_autoregressive\\_image/](https://www.reddit.com/r/LocalLLaMA/comments/1jr6c8e/luminamgpt_20_standalone_autoregressive_image/)
20. OmniParser for pure vision-based GUI agent - Microsoft Research, accessed December 20, 2025, <https://www.microsoft.com/en-us/research/articles/omniparser-for-pure-vision-based-gui-agent/>
21. E2B (Python) MCP Server: An AI Engineer's Deep Dive, accessed December 20, 2025, <https://skywork.ai/skypage/en/ai-engineer-deep-dive/1978019564182491136>
22. Overview - OpenHands Docs, accessed December 20, 2025, <https://docs.openhands.dev/openhands/usage/runtimes/overview>
23. Rate limits - Documentation - E2B, accessed December 20, 2025, <https://e2b.dev/docs/sandbox/rate-limits>
24. Inspiron 15 3000 (3593) upgrade to 32 GB RAM? - Dell Technologies, accessed December 20, 2025, <https://www.dell.com/community/en/conversations/inspiron/inspiron-15-3000-3593-upgradeto-32-gb-ram/647f84d5f4ccf8a8de3bb1b4>

25. Silicon Power DDR4 3200MT/s (PC4-25600) 8GB-32GB Single Pack 1.2V Laptop SODIMM, accessed December 20, 2025,  
<https://retrogamingofdenver.com/products/silicon-power-ddr4-3200mhz-pc4-25600-8gb-32gb-single-pack-1-2v-laptop-sodimm>
26. Team Group DDR4 SDRAM 32 GB Total Capacity Memory (RAM) for sale | eBay, accessed December 20, 2025,  
[https://www.ebay.com/b/Team-Group-DDR4-SDRAM-32-GB-Total-Capacity-Memory-RAM/170083/bn\\_113557235](https://www.ebay.com/b/Team-Group-DDR4-SDRAM-32-GB-Total-Capacity-Memory-RAM/170083/bn_113557235)